

JavaScript Asynchronous Programming

24.1 — Callbacks & Async Foundation

Goal: এই document শেষ করার পর তুমি বুঝবে JavaScript কীভাবে synchronous code execute করে, asynchronous কাজ কোথায় যায়, callback কী, Event Loop কীভাবে কাজ করে এবং কেন Callback Hell থেকে Promise-এর প্রয়োজন হয়েছিল।

1. Synchronous vs Asynchronous JavaScript

JavaScript-এর default execution model হলো **synchronous**।

মানে JavaScript সাধারণত code একটার পর একটা, উপরের দিক থেকে নিচের দিকে execute করে।

Synchronous Example

```
console.log("A");  
console.log("B");  
console.log("C");
```

Output:

```
A  
B  
C
```

এখানে:

```
A শেষ  
↓  
B শুরু  
↓  
B শেষ  
↓  
C শুরু
```

একটা statement শেষ হওয়ার আগে পরের statement execute হয় না।

2. Blocking vs Non-Blocking

এখানে একটা important distinction আছে।

Blocking

যদি কোনো কাজ JavaScript-এর পরবর্তী execution আটকে রাখে, তাকে blocking behavior বলা যায়।

ধরো:

```
console.log("Start");

const start = Date.now();

while (Date.now() - start < 3000) {
  // 3 seconds busy
}

console.log("End");
```

এখানে JavaScript প্রায় 3 seconds busy থাকবে।

ততক্ষণ:

End

print হবে না।

Browser-এর main thread-এ এই ধরনের কাজ UI-ও freeze করতে পারে।

Non-blocking

Asynchronous API-এর ক্ষেত্রে JavaScript কাজটা শুরু করে অন্য কাজ চালিয়ে যেতে পারে।

```
console.log("Start");

setTimeout(() => {
  console.log("Timer finished");
}, 3000);
```

```
console.log("End");
```

Output:

```
Start  
End  
Timer finished
```

এখানে 3 seconds অপেক্ষা করে JavaScript বসে থাকেনি।

3. JavaScript কি সত্যিই Asynchronous?

এখানে beginners-এর একটা common confusion আছে।

JavaScript নিজে সাধারণত single-threaded এবং synchronous execution model ব্যবহার করে, কিন্তু JavaScript environment asynchronous operations provide করে।

Browser এবং Node.js-এর মতো host environment JavaScript-এর বাইরে বিভিন্ন asynchronous capability provide করে।

যেমন:

```
JavaScript  
  ↓  
Host Environment  
  ↓  
Timers / Network / File System / Events
```

তাই:

JavaScript language + host environment = practical asynchronous programming

4. Single-Threaded মানে কী?

JavaScript-এর main execution context-এ সাধারণত একটি **Call Stack** থাকে।

সহজভাবে:

Call Stack

এক সময় একটি execution

তাই একই মুহূর্তে JavaScript-এর main thread-এ দুইটা normal JavaScript function পাশাপাশি execute হয় না।

Example:

```
function one() {  
  console.log("One");  
}  
  
function two() {  
  console.log("Two");  
}  
  
one();  
two();
```

Execution:

```
one()  
↓  
console.log("One")  
↓  
one() শেষ  
↓  
two()  
↓  
console.log("Two")
```

5. Call Stack

Call Stack হলো JavaScript execution-এর জন্য ব্যবহৃত stack data structure।

যখন function call হয়, সেটা stack-এ যায়।

```
function first() {  
  second();  
}
```

```
function second() {  
  third();  
}  
  
function third() {  
  console.log("Hello");  
}  
  
first();
```

Conceptually:

```
third()  
second()  
first()  
global()
```

`third()` শেষ হলে সেটা stack থেকে বের হয়।

তারপর:

```
second()  
first()  
global()
```

এভাবে execution শেষ হয়।

6. Stack কেন Important?

Asynchronous JavaScript বুঝতে হলে Call Stack বুঝতেই হবে।

কারণ Event Loop মূলত দেখে:

"Call Stack এখন empty কি না?"

তারপর queued asynchronous callbacks-এর মধ্যে কোনটা execution-এর জন্য নেওয়া যাবে সেটা নির্ধারণে সাহায্য করে।

7. Callback Function কী?

যে function-কে অন্য একটা function-এর argument হিসেবে pass করা হয় এবং পরে call করা হয়, তাকে সাধারণভাবে **callback function** বলা হয়।

Example:

```
function greet(name, callback) {  
  console.log("Hello", name);  
  
  callback();  
}  
  
function done() {  
  console.log("Finished");  
}  
  
greet("Shariar", done);
```

Output:

```
Hello Shariar  
Finished
```

এখানে:

```
done
```

হলো callback।

8. Function Pass করা vs Function Call করা

এটা খুব important।

Function pass করা

```
greet("Shariar", done);
```

এখানে `done` pass করা হয়েছে।

Function call করা

```
greet("Sharir", done());
```

এখানে `done()` immediately execute হবে।

সুতরাং callback-এর ক্ষেত্রে সাধারণত:

```
callback
```

pass করবে, `callback()` নয়।

9. Callback কেন ব্যবহার করি?

Callback-এর একটি major purpose হলো:

"কোনো কাজ শেষ হলে এই function-টা execute করে।"

Example:

```
function fetchData(callback) {  
  setTimeout(() => {  
    const data = {  
      id: 1,  
      name: "Sharir",  
    };  
  
    callback(data);  
  }, 2000);  
}  
  
fetchData((data) => {  
  console.log(data);  
});
```

এখানে:

```
fetchData()  
  ↓  
2 seconds  
  ↓
```

```
data ready
  ↓
callback(data)
```

10. Callback শুধু Async-এর জন্য নয়

Callback asynchronous programming-এ খুব common হলেও callback মানেই asynchronous নয়।

Example:

```
const numbers = [1, 2, 3];

numbers.forEach((number) => {
  console.log(number);
});
```

এখানে callback আছে:

```
(number) => {
  console.log(number);
}
```

কিন্তু `forEach()` নিজে asynchronous নয়।

অর্থাৎ:

Callback $\neq$ automatically asynchronous

Callback synchronous-ও হতে পারে, asynchronous-ও হতে পারে।

11. Synchronous Callback

```
function process(callback) {
  console.log("Before");

  callback();

  console.log("After");
}
```



```
process(() => {  
  console.log("Callback");  
});
```

Output:

```
Before  
Callback  
After
```

Callback immediately execute হয়েছে।

12. Asynchronous Callback

```
function process(callback) {  
  console.log("Before");  
  
  setTimeout(() => {  
    callback();  
  }, 1000);  
  
  console.log("After");  
}  
  
process(() => {  
  console.log("Callback");  
});
```

Output:

```
Before  
After  
Callback
```

কারণ callback-এর execution asynchronous timer-এর মাধ্যমে পরে হয়েছে।

13. Callback-based Async API

পুরোনো JavaScript ecosystem-এ asynchronous কাজ অনেক সময় callback দিয়ে handle করা হতো।

Conceptual example:

```
getData((error, data) => {  
  if (error) {  
    console.error(error);  
    return;  
  }  
  
  console.log(data);  
});
```

এখানে callback দুই ধরনের information পেতে পারে:

```
error  
data
```

14. Error-First Callback Pattern

Node.js ecosystem-এ ঐতিহাসিকভাবে খুব common pattern হলো:

```
callback(error, result);
```

Example:

```
function divide(a, b, callback) {  
  if (b === 0) {  
    callback(new Error("Cannot divide by zero"), null);  
    return;  
  }  
  
  callback(null, a / b);  
}
```

Usage:

```
divide(10, 2, (error, result) => {  
  if (error) {  
    console.error(error);  
    return;  
  }  
  
  console.log(result);  
});
```

Successful case:

```
error → null  
result → 5
```

Error case:

```
error → Error object  
result → null
```

15. Why Error-First?

এখানে convention হলো:

```
callback(error, result);
```

প্রথম argument:

```
error
```

দ্বিতীয় argument:

```
result
```

তাই callback সাধারণত:

```
if (error) {  
  // handle error
```

```
    return;  
}  
  
console.log(result);
```

এই pattern Promise আসার আগে asynchronous error handling-এর খুব important approach ছিল।

16. Asynchronous Callback-এর Real Example

ধরো একটা API:

```
function getUser(callback) {  
  setTimeout(() => {  
    callback(null, {  
      id: 1,  
      name: "Shariar",  
    });  
  }, 1000);  
}
```

Use:

```
getUser((error, user) => {  
  if (error) {  
    console.error(error);  
    return;  
  }  
  
  console.log(user);  
});
```

17. Multiple Async Operations

এখন ধরো:

1. User fetch
2. তারপর user's posts fetch
3. তারপর posts-এর comments fetch

Callback দিয়ে:

```
getUser((error, user) => {  
  if (error) {  
    console.error(error);  
    return;  
  }  
  
  getPosts(user.id, (error, posts) => {  
    if (error) {  
      console.error(error);  
      return;  
    }  
  
    getComments(posts[0].id, (error, comments) => {  
      if (error) {  
        console.error(error);  
        return;  
      }  
  
      console.log(comments);  
    });  
  });  
});
```

এখান থেকেই সমস্যা শুরু হয়।

18. Callback Hell

অনেক nested callback একটার ভেতরে আরেকটা ঢুকে গেলে code খুব difficult হয়ে যায়।

এটাকে বলা হয়:

Callback Hell

Visual:

```
getUser()  
↓  
callback  
↓  
getPosts()  
↓
```

```
callback
  ↓
getComments()
  ↓
callback
  ↓
getLikes()
  ↓
callback
```

Code দেখতে এমন হতে পারে:

```
getUser((error, user) => {
  if (error) return handleError(error);

  getPosts(user.id, (error, posts) => {
    if (error) return handleError(error);

    getComments(posts[0].id, (error, comments) => {
      if (error) return handleError(error);

      getLikes(comments[0].id, (error, likes) => {
        if (error) return handleError(error);

        console.log(likes);
      });
    });
  });
});
```

19. Callback Hell-এর Problems

1. Readability কমে যায়

Code deeply nested হয়।

2. Error handling repetitive হয়

প্রতিটা level-এ:

```
if (error) {  
  return handleError(error);  
}
```

লিখতে হয়।

3. Maintain করা কঠিন

একটা step change করলে পুরো chain-এর structure affect হতে পারে।

4. Testing কঠিন

Nested callbacks test করা comparatively difficult।

5. Control flow বুঝতে সমস্যা হয়

Code দেখতে নিচের দিকে যাচ্ছে:

```
↓  
↓  
↓  
↓
```

কিন্তু logical flow complex হয়ে যায়।

20. Inversion of Control

Callback-based programming-এর একটা deeper problem হলো **Inversion of Control**।

ধরো:

```
someAsyncOperation(() => {  
  console.log("Done");  
});
```

তুমি callback function অন্য একটা function-এর হাতে দিয়ে দিলে।

এখন প্রশ্ন:

কে guarantee করবে callback ঠিক একবার execute হবে?

অথবা:

- আদৌ execute করবে?
- দুইবার execute করবে?
- ভুল সময়ে execute করবে?
- error হলে callback করবে?

Example:

```
function badAPI(callback) {  
  callback();  
  callback();  
}
```

এখন:

```
badAPI(() => {  
  console.log("Done");  
});
```

Output:

```
Done  
Done
```

তুমি হয়তো একবার আশা করেছিলে।

এই control problem Promise design-এর একটা বড় motivation ছিল।

21. Callback-এর আরও সমস্যা

ধরো callback API:

```
doSomething((error, result) => {  
  // ...  
});
```

তাহলে তোমাকে API provider-এর convention follow করতে হবে।

কিন্তু Promise:


```
doSomething()  
  .then(...)  
  .catch(...);
```

একটা standardized abstraction দেয়।

22. Callback থেকে Promise-এর দিকে কেন গেলাম?

Callback:

```
getUser((error, user) => {  
  if (error) {  
    handleError(error);  
    return;  
  }  
  
  getPosts(user.id, (error, posts) => {  
    if (error) {  
      handleError(error);  
      return;  
    }  
  
    console.log(posts);  
  });  
});
```

Promise:

```
getUser()  
  .then((user) => getPosts(user.id))  
  .then((posts) => {  
    console.log(posts);  
  })  
  .catch(handleError);
```

এখানে:

```
Callback Hell  
  ↓  
Promise  
  ↓
```

Promise Chaining

↓

async/await

এই progression পুরো Async JavaScript শেখার backbone ।

23. Event Loop — Async JavaScript-এর Heart

এখন সবচেয়ে important অংশ ।

JavaScript-এর execution model বুঝতে broadly এই components মনে রাখো:



এটা conceptual model ।

Browser এবং Node.js-এর actual internals আরও complex ।

24. `setTimeout()` আসলে কী করে?

অনেকে ভাবে:

```
setTimeout(fn, 2000);
```

মানে:

"2 seconds পরে অবশ্যই function execute হবে।"

এটা পুরোপুরি correct নয়।

বরং:

কমপক্ষে **specified delay**-এর পরে **callback execution**-এর জন্য **eligible** হবে।

Example:

```
console.log("A");

setTimeout(() => {
  console.log("B");
}, 0);

console.log("C");
```

Output:

```
A
C
B
```

0ms মানে immediate না।

25. কেন B শেষে?

Step-by-step:

Step 1

```
console.log("A");
```

Call Stack-এ execute হলো।

A

Step 2

```
setTimeout(...)
```

Timer setup হলো।

Callback পরে queue-এর দিকে যাবে।

Step 3

```
console.log("C");
```

synchronous code আগে শেষ হলো।

A
C

Step 4

Stack empty হলে queued callback execution-এর সুযোগ পায়।

B

Final:

A
C
B

26. Event Loop-এর Basic Rule

Beginner level mental model:

Synchronous code আগে শেষ হবে। তারপর **eligible asynchronous callbacks execute** হওয়ার সুযোগ পাবে।

এটা পরে Promise-এর ক্ষেত্রে আরও refine করতে হবে, কারণ Promise callbacks **microtask queue** ব্যবহার করে।

27. Callback Queue

Timer callback-এর মতো asynchronous callbacks execution-এর জন্য queue-তে অপেক্ষা করতে পারে।

Conceptually:

```
Call Stack
  ↓
Empty?
  ↓
Queue থেকে callback
  ↓
Call Stack
```

Event Loop এই process coordinate করতে সাহায্য করে।

28. Callback Queue বনাম Microtask Queue

এখন শুধু basic ধারণা রাখো।

সাধারণ **task**-এর **example**

```
setTimeout(...)
```

Microtask-এর **example**

```
Promise.resolve().then(...)
```

পরের Promise document-এ আমরা এটা deep dive করব।

Basic mental model:

```
Synchronous JavaScript
  ↓
Microtasks
```

↓
Tasks

এই ordering-এর exact rules আমরা পরে বিস্তারিত দেখব।

29. Classic Event Loop Example

```
console.log("Start");  
  
setTimeout(() => {  
  console.log("Timeout");  
}, 0);  
  
console.log("End");
```

Output:

```
Start  
End  
Timeout
```

কারণ:

Start	→ synchronous
End	→ synchronous
Timeout	→ asynchronous callback

30. Callback + Timer

```
function greet(callback) {  
  setTimeout(() => {  
    callback("Hello");  
  }, 1000);  
}  
  
greet((message) => {  
  console.log(message);  
});
```

```
console.log("Waiting...");
```

Output:

```
Waiting...  
Hello
```

Flow:

```
greet()  
  ↓  
timer registered  
  ↓  
"Waiting..." executes  
  ↓  
timer completes  
  ↓  
callback executes
```

31. Callback Hell Refactoring

Callback hell:

```
getUser((error, user) => {  
  if (error) return handleError(error);  
  
  getPosts(user.id, (error, posts) => {  
    if (error) return handleError(error);  
  
    getComments(posts[0].id, (error, comments) => {  
      if (error) return handleError(error);  
  
      console.log(comments);  
    });  
  });  
});
```

একটা ছোট improvement হলো named functions ব্যবহার করা।

```

function handleUser(error, user) {
  if (error) {
    return handleError(error);
  }

  getPosts(user.id, handlePosts);
}

function handlePosts(error, posts) {
  if (error) {
    return handleError(error);
  }

  getComments(posts[0].id, handleComments);
}

function handleComments(error, comments) {
  if (error) {
    return handleError(error);
  }

  console.log(comments);
}

getUser(handleUser);

```

Nested structure কমেছে।

তবে problem পুরোপুরি solve হয়নি।

এ কারণেই Promise আরও useful abstraction দেয়।

32. Callback Hell → Promise

ধরো:

```

function getUser() {
  return new Promise((resolve, reject) => {
    // ...
  });
}

```

তারপর:


```

getUser()
  .then((user) => getPosts(user.id))
  .then((posts) => getComments(posts[0].id))
  .then((comments) => {
    console.log(comments);
  })
  .catch(handleError);

```

এখন control flow অনেক পরিষ্কার।

আর পরের ধাপে:

```

async function loadData() {
  const user = await getUser();
  const posts = await getPosts(user.id);
  const comments = await getComments(posts[0].id);

  return comments;
}

```

এটাই async/await-এর দিকে progression।

33. Callback vs Promise vs async/await

Feature	Callback	Promise	async/await
Async handling	✓	✓	✓
Nested code	বেশি হতে পারে	কম	সাধারণত সবচেয়ে readable
Error handling	convention-based	<code>.catch()</code>	<code>try/catch</code>
Chaining	কঠিন	সহজ	সহজ
Composition	সীমিত	শক্তিশালী	Promise-এর ওপর ভিত্তি করে
Modern code	কম preferred	Common	Very common

Important:

`async/await` Promise-এর replacement নয়।

বরং:

async/await

↓

Promises -এর ওপর কাজ করে

34. Common Mistakes

Mistake 1 — Callback immediately call করা

Wrong:

```
setTimeout(doSomething(), 1000);
```

Correct:

```
setTimeout(doSomething, 1000);
```

অথবা:

```
setTimeout(() => {  
  doSomething();  
}, 1000);
```

Mistake 2 — Callback মানেই Async ভাবা

Wrong assumption:

```
array.forEach(callback);
```

মানে asynchronous ।

না।

`forEach()` synchronous ।

Mistake 3 — `setTimeout(..., 0)` মানে **immediately**

না।

```
setTimeout(fn, 0);
```

মানে callback পরে execute করার জন্য schedule করা।

Mistake 4 — Event Loop JavaScript-কে **multi-threaded** বানায়

না।

Event Loop asynchronous coordination-এর অংশ।

এটা simply:

"JavaScript main execution-এর পাশাপাশি host environment-এর async কাজগুলো কীভাবে coordinate হবে"

এটা বুঝতে সাহায্য করে।

35. Real-World Example — API Request

ধরো user button click করলো:

```
User clicks button
      ↓
JavaScript function
      ↓
API request starts
      ↓
JavaScript অন্য কাজ করতে পারে
      ↓
Server response আসে
      ↓
Callback/Promise continuation
      ↓
UI update
```

এখানে asynchronous programming-এর মূল benefit:

Network response-এর জন্য পুরো application execution বসে থাকে না।

36. Real-World Example — Search

ধরো user লিখছে:

```
javascript
```

প্রতিবার input change হলে API request হতে পারে।

```
j      → request
ja     → request
jav    → request
java   → request
javasc → request
...    
```

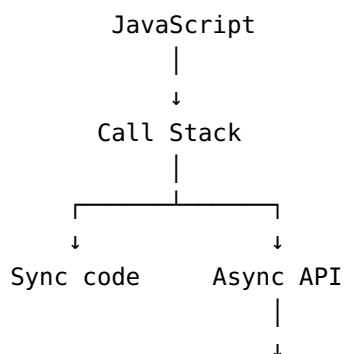
এখানে advanced async concepts পরে কাজে আসবে:

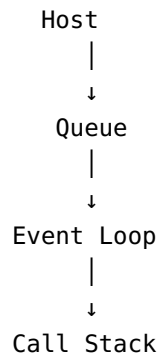
- debounce
- cancellation
- AbortController
- race conditions

এসব আমরা Advanced Async Patterns-এ cover করব।

37. Mental Model

Async JavaScript আপাতত এভাবে ভাবো:





আর পুরো learning progression:

```
Callbacks
  ↓
Callback Hell
  ↓
Promises
  ↓
Promise Chaining
  ↓
async/await
  ↓
Concurrency
  ↓
Advanced Async Patterns
```

38. Practice — Level 1

Question 1

Callback function কী?

নিজের ভাষায় explain করো এবং একটা example লেখো।

Question 2

নিচের code-এ callback কোনটা?

```
function process(value, callback) {
  callback(value * 2);
```

```
}  
  
process(5, (result) => {  
  console.log(result);  
});
```

Question 3

Output কী?

```
console.log("A");  
  
setTimeout(() => {  
  console.log("B");  
}, 0);  
  
console.log("C");
```

Question 4

Callback কি সবসময় asynchronous?

Example দিয়ে explain করো।

39. Practice — Level 2

Question 5

একটা function লেখো:

```
delay(2000, callback);
```

যেটা 2 seconds পরে callback execute করবে।

Question 6

এই callback API-টা implement করো:

```
divide(10, 2, (error, result) => {  
  if (error) {  
    console.log(error);  
    return;  
  }  
  
  console.log(result);  
});
```

`b === 0` হলে error callback করবে।

Question 7

নিচের code-টা callback hell থেকে named functions ব্যবহার করে refactor করো:

```
getUser((error, user) => {  
  if (error) return handleError(error);  
  
  getPosts(user.id, (error, posts) => {  
    if (error) return handleError(error);  
  
    getComments(posts[0].id, (error, comments) => {  
      if (error) return handleError(error);  
  
      console.log(comments);  
    });  
  });  
});
```

40. Practice — Level 3: Output Prediction

Question 8

```
console.log(1);  
  
setTimeout(() => {  
  console.log(2);  
}, 1000);  
  
console.log(3);
```

Output কী?

Question 9

```
function test(callback) {  
  console.log("A");  
  
  setTimeout(() => {  
    callback();  
  }, 0);  
  
  console.log("B");  
}  
  
test(() => {  
  console.log("C");  
});  
  
console.log("D");
```

Output predict করো।

Question 10

```
console.log("Start");  
  
setTimeout(() => {  
  console.log("Timer");  
}, 0);  
  
function hello() {  
  console.log("Hello");  
}  
  
hello();  
  
console.log("End");
```

Output predict করো এবং Call Stack-এর execution explain করো।

41. Practice — Level 4: Tricky

Question 11

কেন এটা ভুল?

```
setTimeout(console.log("Hello"), 1000);
```

Correct version লেখো।

Question 12

এই দুইটার difference explain করো:

```
setTimeout(callback, 1000);
```

vs

```
setTimeout(callback(), 1000);
```

Question 13

`setTimeout(fn, 0)` কি guarantee করে যে `fn` ঠিক 0 milliseconds পরে execute হবে?

Explain করো।

Question 14

JavaScript single-threaded হওয়া সত্ত্বেও network request চলাকালীন application কীভাবে অন্য JavaScript কাজ করতে পারে?

নিজের mental model দিয়ে explain করো।

42. Challenge — Callback-based API

একটা fake API তৈরি করো:

```
getUser(id, callback);
```

যেখানে:

- 1 second delay থাকবে
- user পাওয়া গেলে user object return করবে
- user না পাওয়া গেলে error return করবে
- error-first callback pattern ব্যবহার করবে

Expected usage:

```
getUser(1, (error, user) => {  
  if (error) {  
    console.error(error);  
    return;  
  }  
  
  console.log(user);  
});
```

43. Challenge — Callback Chain

এই flow implement করো:

```
getUser()  
  ↓  
getPosts()  
  ↓  
getComments()
```

প্রতিটা function:

- asynchronous হবে
- callback ব্যবহার করবে
- error-first pattern follow করবে

তারপর intentionally callback hell তৈরি করো।

এরপর একই code named functions দিয়ে refactor করো।

পরের **document**-এ আমরা এটাকেই **Promise** দিয়ে আরও সুন্দরভাবে **solve** করব।

44. Interview Questions

Basic

1. JavaScript synchronous না asynchronous?
2. Single-threaded বলতে কী বোঝায়?
3. Call Stack কী?
4. Callback function কী?
5. Callback এবং asynchronous function-এর মধ্যে difference কী?
6. Blocking vs non-blocking কী?
7. `setTimeout(fn, 0)` কী করে?

Intermediate

1. Callback Hell কী?
2. Callback Hell-এর problems কী?
3. Error-first callback pattern কী?
4. Inversion of Control কী?
5. Promise কেন callback-এর তুলনায় better abstraction?
6. Event Loop কীভাবে কাজ করে?

Advanced Foundation

1. JavaScript single-threaded হয়েও asynchronous কীভাবে?
2. Browser-এর role কী?
3. `setTimeout()` কি JavaScript-এর language feature?
4. Event Loop কি JavaScript engine-এর একমাত্র responsibility?
5. Callback queue কী?
6. Microtask এবং task-এর difference কী?

45. Quick Revision

Synchronous

→ code one by one execute হয়

Blocking

→ execution আটকে রাখে

Non-blocking

→ async operation-এর সময় অন্য কাজ চলতে পারে

Callback

→ অন্য function-এ pass করা function

Callback Hell

→ deeply nested callbacks

Error-first callback

→ `callback(error, result)`

Call Stack

→ JavaScript execution stack

Event Loop

→ queued async work-এর continuation execution coordinate করে

`setTimeout`

→ callback-কে delay-এর পর execution-এর জন্য eligible করে

Promise

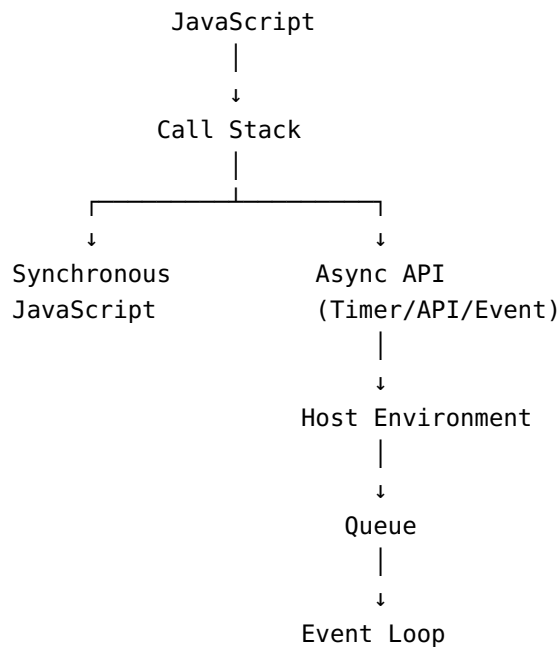
→ callback-based async handling-এর আরও structured abstraction

`async/await`

→ Promise-based async code লেখার cleaner syntax

46. Final Mental Model

সবচেয়ে important এই flow-টা মনে রাখো:



|
↓
Call Stack

তারপর evolution:

Callback
↓
Callback Hell
↓
Promise
↓
Promise Chaining
↓
async/await
↓
Concurrency
↓
Advanced Async Patterns

✓ Mastery Checklist

এই document শেষ করার পর নিশ্চিত হও:

- ☐ Synchronous কী বুঝি
- ☐ Asynchronous কী বুঝি
- ☐ Blocking vs non-blocking বুঝি
- ☐ JavaScript single-threaded concept বুঝি
- ☐ Call Stack বুঝি
- ☐ Callback কী বুঝি
- ☐ Callback synchronous হতে পারে বুঝি
- ☐ Callback asynchronous হতে পারে বুঝি
- ☐ Error-first callback বুঝি
- ☐ Callback Hell বুঝি
- ☐ Inversion of Control বুঝি
- ☐ `setTimeout()` কীভাবে কাজ করে বুঝি
- ☐ `setTimeout(fn, 0)` immediate নয় বুঝি
- ☐ Event Loop-এর basic model বুঝি
- ☐ Queue-এর basic ধারণা আছে
- ☐ Callback → Promise → async/await progression বুঝি
- ☐ Basic async output prediction করতে পারি

Next: 24.2 – Promises Deep Dive

এখানে আমরা Promise-এর **pending** → **fulfilled/rejected**, **resolve/reject**, **.then()**, **.catch()**, **.finally()**, chaining, error propagation এবং Promise-এর ভিতরের mental model একদম deepভাবে শিখব।